Week 16 (Days 106-112): Preference tuning: RLHF, reward models, DPO and evaluation pitfalls

Status: written in full (Phase 4). This week replaces the older Week 16 plan (embeddings, RAG, prompting, quantisation, KV cache), which moves to later weeks (see `SYLLABUS.md`). Every number below was printed by a script in `code/week-16/` on this box (torch 2.15.0+cpu, transformers 5.21.1, 8 CPU threads, no GPU), with logs in `code/week-16/out/*.log`. Scripts were run one at a time. Shared helpers are in `code/week-16/_pref.py`, which reuses `code/week-15/_hf.py` and `_lora.py`.

Week goals

- Understand the RLHF pipeline: supervised fine-tuning, a reward model from pairwise preferences, then KL-regularised policy optimisation.
- Fit a Bradley-Terry reward model, first on a toy problem and then on a small language model, and watch it learn a shortcut.
- Derive and implement Direct Preference Optimisation (DPO) from scratch on SmolLM2-135M-Instruct with LoRA.
- See what ? and the learning rate do, and why "preference accuracy = 1.00" does not mean the model is good.
- Measure evaluation pitfalls: small test sets, prompt wording, metric choice, contamination.
- Use inference-time selection (best-of-n) and see a reward being exploited.
- Weekly project: SFT vs DPO vs DPO+NLL on the same data and budget.

```
flowchart LR
  A[Day 106 RLHF toys] --> B[Day 107 reward model]
  B --> C[Day 108 DPO from scratch]
  C --> D[Day 109 beta and lr sweep]
  D --> E[Day 110 eval pitfalls]
  E --> F[Day 111 best-of-n]
  F --> G[Day 112 project + quiz]
```

Compute note (CPU vs GPU). All language-model work uses SmolLM2-135M(-Instruct), LoRA r=8 or r=16, batches of 4-8 *pairs* and 60-120 steps. One DPO step (4 pairs, so 8 sequences forward and backward) took about 0.6-0.7 s here (120 steps in 84 s; 60 steps in 35-36 s). That shows the *mechanics* and the failure modes. It does not reproduce real preference tuning, which uses tens of thousands of human or AI-labelled pairs, models of 1-70B parameters, and GPUs. On a GPU you would also run PPO properly (it needs a policy, a reference, a reward model and a value model in memory at once, plus sampling in the loop), which is not practical on this CPU.

Models and data. `HuggingFaceTB/SmolLM2-135M` and `HuggingFaceTB/SmolLM2-135M-Instruct` from the local Hugging Face cache (verified in Week 15). The preference data is **synthetic and generated by the scripts**: prompts "Reverse the order of these words: ..." over a fixed 40-word vocabulary, 1,000 training prompts and 40 held-out test prompts (test prompts that also occur in training are removed). *Chosen* = the correctly reversed words; *rejected* = the words left in their original order (or a random other order where stated). Synthetic pairs make the "true" preference unambiguous, which is what we need to see the algorithms clearly; real preference data is noisy and subjective.

Day 106: RLHF overview: preferences, Bradley-Terry and the KL-regularised objective

Review today: Day 105 (+1), Day 103 (+3), Day 99 (+7), Day 85 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Why preferences? After pretraining a model predicts text; after supervised fine-tuning (SFT) on demonstrations it follows instructions. But many qualities we want (helpful, harmless, concise, honest) are easier to *judge* than to *demonstrate*. A labeller who cannot write the perfect answer can still say which of two answers is better. RLHF (reinforcement learning from human feedback) turns those comparisons into a training signal.

The classic three-stage pipeline. 1. *SFT*: fine-tune the pretrained model on demonstrations. This is the starting policy and usually also the *reference* policy pi_ref. 2. *Reward model (RM)*: collect pairs (prompt x, answer y_w preferred over y_l) and train a model r(x, y) that gives the preferred answer a higher score. 3. *Policy optimisation*: sample answers from the policy, score them

with the RM, and update the policy to get higher reward, while a KL penalty keeps it close to pi_ref. The usual optimiser is PPO (proximal policy optimisation).

Bradley-Terry. The standard link between rewards and preferences is P(y_w ? y_l | x) = sigma(r(x, y_w) - r(x, y_l)), where sigma is the logistic sigmoid. Training the RM is then logistic regression on reward *differences*: loss = -log sigma(r_w - r_l). Two consequences: only differences matter, so the reward is defined only up to a constant per prompt; and the scale is set by how confident the labels are.

The KL-regularised objective. We maximise E_{y?pi}[r(x, y)] - ?.KL(pi || pi_ref). Without the KL term the policy would collapse onto whatever the RM over-rates, including its mistakes (reward hacking). ? trades reward against staying close to a model we trust. For a fixed prompt this objective has a closed-form optimum: pi*(y) ? pi_ref(y).exp(r(y)/?). Small ? gives a nearly greedy policy; large ? leaves you at pi_ref. Remember this formula: DPO on Day 108 is derived from it.

PPO intuition. PPO estimates how much better than expected each sampled answer was (the *advantage*), raises the log-probability of good samples and lowers bad ones, and clips the update so the new policy does not move too far from the policy that collected the samples in one step. In LLM RLHF the reward is usually given once per full answer, and the KL penalty is added per token. A value network (critic) estimates expected reward to reduce variance. That makes four models in memory at once, which is why simpler methods (DPO, and critic-free methods like REINFORCE-style or group-relative baselines) became popular.

Block 2: Worked example (45 min)

`code/week-16/day106_rlhf_toy.py` has two parts, both pure torch and done in a few seconds.

Part 1: Bradley-Terry. A hidden linear reward r(x) = w.x with w = [2.0, -1.0, 0.5, 0.0, 1.5] generates 2,000 noisy preferences (sampled with P = sigma(r_a - r_b)). Fitting w by logistic loss on x_a - x_b recovers it.

Part 2: KL-regularised bandit. Five arms with rewards [1.0, 0.8, 0.5, 0.2, 0.0] and a reference policy [0.05, 0.1, 0.2, 0.3, 0.35] that prefers the *bad* arms. We compute pi* for several ?, then check that REINFORCE with a per-sample KL penalty converges to the same answer.

Verified: - BT fit: fitted w = [1.92, -1.0, 0.47, 0.05, 1.46]; on 1,000 new pairs it ranks pairs the same way as the true reward in 0.996 of cases; final train loss 0.299. - Adding +10 to every reward changes the BT loss by 0.0e+00: the constant is not identifiable. - ? = 0.05: pi* = [0.964, 0.035, 0.0, 0.0, 0.0], expected reward 0.993, KL to ref 2.816. - ? = 0.2: pi* = [0.45, 0.331, 0.148, 0.049, 0.021], reward 0.799, KL 1.193. - ? = 1.0: pi* = [0.097, 0.158, 0.235, 0.261, 0.249], reward 0.393, KL 0.053. - ? = 5.0: pi* = [0.058, 0.111, 0.208, 0.294, 0.33], reward 0.309, KL 0.002. - REINFORCE with KL penalty (? 0.2, 3,000 steps x 64 samples) ended at [0.45, 0.331, 0.148, 0.049, 0.021], equal to the closed form to three decimals.

Read the ? table as the whole RLHF trade-off in one place: reward rises from 0.309 to 0.993 as ? falls, and the price is KL rising from 0.002 to 2.816.

Block 3: Lab (2h)

1. Run the script and confirm the numbers above.
2. Change N from 2,000 to 200 and to 20,000. How close is the fitted w each time? (Expect the error to shrink roughly like 1/sqrtN.)
3. Multiply w_true by 0.25 (noisier labels). The fitted w should also shrink: BT learns the reward *in units of label confidence*.
4. In part 2, set the reference to uniform. How does pi* change for ? = 0.2?
5. Remove the KL term from the REINFORCE advantage. Where does the policy end up? Why is that dangerous if r were a learned, imperfect reward model?
6. Write down, in your own words, why the KL term is computed against pi_ref and not against the previous policy.

Block 4: Practice (1h)

1. Show that sigma(a) + sigma(-a) = 1, and explain why that means BT treats (y_w, y_l) and (y_l, y_w) consistently.
2. Derive pi* ? pi_ref.exp(r/?) by maximising Sum pi(y) r(y) - ? Sum pi(y) log(pi(y)/pi_ref(y)) subject to Sum pi = 1 (use a Lagrange multiplier).
3. For ? -> 0 and ? -> inf, what does pi* become?
4. Why does the BT loss not change when you add a constant to all rewards?

Answers. (1) sigma(-a) = 1/(1+e^{a}) = 1 - sigma(a); swapping the pair flips the sign of the difference, so the probabilities stay complementary. (2) Setting the derivative to zero gives r(y) - ?(log(pi/pi_ref) + 1) - lambda = 0, so pi ? pi_ref.exp(r/?). (3) ? -> 0: all mass on the best arm (argmax r); ? -> inf: pi_ref. (4) The loss uses only r_w - r_l; the constant cancels (measured 0.0e+00 change).

Block 5: Review (1h)

Self-check: - [ ] I can name the three RLHF stages and what each consumes and produces. - [ ] I can write the Bradley-Terry likelihood and the RM loss. - [ ] I can write the KL-regularised objective and its closed-form optimum. - [ ] I can explain the ? trade-off using the Day 106 table. - [ ] I can explain in two sentences what PPO adds on top of plain policy gradient.

Extra notes

The bandit is a "one-step" problem: no sequence of tokens, no credit assignment. Real LLM RLHF is the same objective over whole answers, where pi(y|x) is a product over tokens, so log pi(y|x) is a *sum* of token log-probs. That sum is what Day 108 computes in `logp_batch`. The per-sample KL estimate used in part 2 (log pi(y)/pi_ref(y) for a sampled y) is the same estimator used per token in many LLM RLHF implementations.

Going deeper

- Read the original "Training language models to follow instructions with human feedback" (InstructGPT) paper's method section and match each stage to this day's vocabulary. Find it via a search for the title; check you have the arXiv version.
- Read the PPO paper (Schulman et al., 2017) and find the clipped objective.
- Look up "Constitutional AI" for the idea of using AI-written preference labels (RLAIF).

Interview-style question (15 min)

"Why do RLHF pipelines include a KL penalty to the SFT model? What goes wrong if ? is too small or too large?" A good answer covers: the RM is only accurate near the data it was trained on; without the KL the policy finds inputs where the RM is wrong (reward hacking) and drifts in style or fluency; too-large ? means nothing changes (Day 106: ? = 5.0 moves KL only 0.002); too-small ? means near-greedy optimisation of an imperfect reward.

Common mistakes

- Treating RM scores as absolute. They are only meaningful as differences for the same prompt.
- Thinking the KL is a regulariser on the *weights*. It is on the output *distributions*.
- Forgetting that RLHF does not add knowledge; it reshapes which behaviours the model prefers.
- Calling any fine-tune on preference data "RLHF". DPO, for example, has no RL loop.

What to do next

Tomorrow you train a reward model on a real (small) language model and test whether it learned the rule you intended or a shortcut.

Day 107: Reward modelling on a small LM, and shortcuts

Review today: Day 106 (+1), Day 104 (+3), Day 100 (+7), Day 86 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Architecture. A reward model is usually the SFT model with its language-model head replaced by a *scalar head*: a linear layer from the final hidden state of the *last token* to one number. The whole prompt + answer is read, and the last position summarises it. Training uses the BT loss from Day 106 on (prompt, chosen, rejected) triples. In practice people fine-tune the full network or add LoRA; here we add LoRA r=8 to every linear layer of SmolLM2-135M plus a fresh head (2,442,817 trainable parameters).

What the RM actually learns. The RM learns whatever feature best separates chosen from rejected *in its training data*. If the data contain a spurious correlation (chosen answers are longer, more polite, use bullet points, end with "!"), the RM can learn that instead of the property you care about. During policy optimisation the policy then produces more of that feature: *reward hacking*.

Length bias is the best-documented real example: human and AI raters tend to prefer longer answers, RMs pick this up, and optimised policies get longer.

Generalisation. An RM is only trusted near its training distribution. If training rejections are always of one kind, the RM may learn "detect that kind of mistake" rather than "detect correctness". A policy being optimised will quickly leave the training distribution, so this matters.

Evaluating an RM. The basic metric is held-out *pairwise accuracy*: the fraction of pairs where r(chosen) > r(rejected). Good practice also tests pairs built differently from the training pairs, and checks for known biases (length, format, sycophancy).

Block 2: Worked example (45 min)

`code/week-16/day107_reward_model.py` trains two RMs for 120 steps x 8 pairs each with AdamW (lr 5e-4).

RM A: rejected = unreversed words. Tested on held-out prompts with (i) the same kind of rejection and (ii) a *random other order* as the rejection, which it never saw.

RM B: a planted shortcut. Every chosen training answer ends with "!", and rejected answers (random other orders) never do. Tested with the "!" as in training, with no "!" anywhere, and with the "!" moved to the wrong answer.

Verified (out/day107_reward_model.log): - RM A trained in 90 s; last batch loss 0.000. - RM A held-out accuracy: 1.00 with the training kind of rejection, but only 0.68 when the rejection is a random other order. - Example rewards from RM A on one test prompt: correct 12.02, unreversed -4.59, correct + " Great question!" 9.29. - RM B trained in 105 s; held-out accuracy 1.00 with "!" as in training, 0.62 with no "!" anywhere, and **0.00** when "!" is moved to the wrong answer.

So RM A partly learned "is this the unreversed order?" rather than "is this correctly reversed?", and RM B learned almost only the "!". A 135M model with 120 steps is a small RM, but the pattern (perfect in-distribution accuracy, poor out-of-distribution accuracy, shortcuts) is the general one.

Block 3: Lab (2h)

1. Run the script (about 3.5 minutes) and check your numbers against the log.
2. Train RM A on a mix: half the rejections unreversed, half random orders. Re-measure the 0.68.
3. Make a *length* shortcut: append " Here you go." to every chosen answer. Then test with the suffix on the rejected answer. Compare with RM B.
4. Print the reward for a correct answer followed by an increasing number of random words. Does reward fall smoothly?
5. Replace the last-token readout with mean pooling over the answer tokens. Does that change the shortcut behaviour?

Block 4: Practice (1h)

1. Why read the reward from the last token rather than the first?
2. RM A gives 12.02 to the correct answer and -4.59 to the unreversed one. What BT probability does that imply?
3. Why is held-out accuracy of 1.00 on same-type pairs not enough evidence that an RM is good?
4. Name two real-world spurious features an RM might learn.

Answers. (1) In a causal LM only the last position has attended to the whole sequence. (2) sigma(12.02 - (-4.59)) = sigma(16.61), essentially 1.0: the RM is extremely confident, which is itself a warning sign of over-fitting synthetic data. (3) Because the policy will produce *different* kinds of answers; RM A falls to 0.68 on a new kind of wrong answer. (4) Length, politeness phrases, markdown formatting, agreeing with the user, confident tone.

Block 5: Review (1h)

Self-check: - [ ] I can describe the RM architecture (backbone + scalar head on the last token). - [ ] I can state the RM loss and the pairwise-accuracy metric. - [ ] I can explain the RM A result (1.00 vs 0.68) and the RM B result (0.00 with the "!" moved). - [ ] I can explain how an RM shortcut becomes reward hacking during policy optimisation.

Extra notes

Real mitigations include more diverse rejected answers, training on pairs that differ in one feature only, ensembles of RMs, length-normalised or length-penalised rewards, and keeping the KL penalty. None of these makes an RM "correct"; they make

specific failures less likely. Training loss of 0.000 on the last batch should also worry you: the RM separates training pairs by huge margins, which is over-confidence, not quality.

Going deeper

- Search for "reward model overoptimization scaling laws" (Gao et al.) to see how true quality first rises and then falls as a policy optimises a proxy RM harder.
- Search for "RewardBench" to see how RMs are compared on standard pair sets.

Interview-style question (15 min)

"Your RM scores 98% on its validation pairs, but after RL the model writes long, padded answers. What happened and what would you do?" Expected points: the validation pairs share the training distribution and the length bias; the policy found the bias; check RM reward against length; add length-controlled pairs, penalise length or normalise reward, raise ?, use an RM ensemble, and evaluate with humans or a held-out judge on length-matched comparisons.

Common mistakes

- Testing the RM only on pairs built like its training pairs.
- Comparing rewards across different prompts.
- Forgetting the padding side: with right padding the "last token" index must come from the attention mask (as in the script), not position -1.
- Reading a training loss of 0 as success.

What to do next

Tomorrow you skip the reward model entirely: DPO optimises the same KL-regularised objective directly from the pairs.

Day 108: DPO from scratch

Review today: Day 107 (+1), Day 105 (+3), Day 101 (+7), Day 87 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

From the closed form to a loss. Day 106 showed that the KL-regularised optimum is pi*(y|x) = pi_ref(y|x).exp(r(x,y)/?)/Z(x). Rearranging gives the reward in terms of the policy: r(x,y) = ?.log(pi*(y|x)/pi_ref(y|x)) + ?.log Z(x). Put that into the Bradley-Terry likelihood. Z(x) depends only on the prompt, so it cancels in r(x,y_w) - r(x,y_l), and we get the **DPO loss**:

L = -log sigma(?.[(log pi_?(y_w|x) - log pi_ref(y_w|x)) - (log pi_?(y_l|x) - log pi_ref(y_l|x))]).

No reward model, no sampling, no value network: just two forward passes per pair through the policy, plus reference log-probs, which can be precomputed once because pi_ref is frozen. The quantity ?.log(pi_?/pi_ref) is called the **implicit reward**. The bracket times ? is the **reward margin**; "preference accuracy" is the fraction of pairs with a positive margin.

What the gradient does. The gradient raises log pi_?(y_w) and lowers log pi_?(y_l), weighted by sigma(-margin): pairs the model already ranks correctly get little weight. Nothing in the loss requires the chosen answer to become *likely*, only *more likely relative to the rejected one than under the reference*. Lowering the rejected answer is often the easiest way to do that. This explains today's main result.

Sequence log-probs. log pi(y|x) is the sum of the log-probs of the answer tokens given the prompt and the earlier answer tokens. The prompt tokens are masked out. In `_pref.logp_batch`, the mask starts at position prompt_len - 1 because logits at position t predict token t+1.

Implementation choices. Policy = SmolLM2-135M-Instruct + LoRA r=16 on all linear layers (Week 15's `_lora.py`). Reference = the same instruct model before adding LoRA. ? = 0.1, lr 5e-4, 120 steps x 4 pairs, each training pair used once.

Block 2: Worked example (45 min)

`code/week-16/day108_dpo.py`.

Verified (out/day108_dpo.log): - Reference log-probs for 480 train + 40 test pairs took 23 s. Mean reference log-prob: chosen -20.2, rejected -7.1. The instruct model finds the *wrong* (unreversed) answer far more likely. - Before DPO: exact match 0.00. The model starts explaining ("To reverse the order of these words, I will first identify the words that are"). - Step 0: loss 0.693 (= log 2, margin 0, as expected at initialisation since LoRA B = 0). - Step 20: loss 0.000, margin 9.87, pref acc 1.00; ?log-prob chosen +10.7, rejected -87.9. - Step 119: margin 12.08; ? chosen +14.2, rejected -106.6. Training took 84 s. - Held-out pairs: implicit-reward preference accuracy 1.00, mean margin 13.53, ? chosen +8.6, ? rejected -126.7. - After DPO: exact match **0.17**. Example: "spring orange lemon cloud" -> "clouds orange springfirehose".

The loss was at 0.000 from step 20, and held-out preference accuracy was perfect, yet only 0.17 of generations were exactly right and the example output is broken. DPO mostly made the rejected answer extremely unlikely (-126.7 nats) while raising the chosen one only modestly. Pushing down one specific wrong answer does not make the right answer the model's greedy output, and with a high learning rate the probability mass goes partly to garbage.

Block 3: Lab (2h)

1. Run the script (about 2 minutes) and compare with the log.
2. Print the full ?log-prob trajectory every 5 steps and plot chosen vs rejected.
3. Make the rejected answer a random other order instead of the unreversed one. Does exact match change?
4. Implement the DPO loss with `torch.nn.functional.logsigmoid` and check against a hand computation for one pair.
5. Verify that at step 0 the loss is exactly log 2 ~= 0.693, and explain why using the LoRA initialisation.
6. Add the reference model as a second copy instead of precomputing its log-probs; time both versions.

Block 4: Practice (1h)

1. Why does Z(x) cancel in the DPO derivation?
2. What does a margin of 12.08 at ? = 0.1 mean in log-prob units?
3. Why can the reference log-probs be computed once before training?
4. Give one reason exact match can stay low when preference accuracy is 1.00.

Answers. (1) It is the same for both answers to the same prompt, and BT uses only the difference. (2) Margin/? = 120.8 nats of relative log-prob change between chosen and rejected. (3) pi_ref is frozen and the pairs are fixed (offline data). (4) The loss is satisfied by lowering the rejected answer; it does not make the chosen answer the argmax of generation, and other unseen outputs can gain mass (measured: ? rejected -126.7 vs ? chosen +8.6).

Block 5: Review (1h)

Self-check: - [ ] I can derive the DPO loss from the Day 106 closed form in five lines. - [ ] I can define implicit reward, reward margin and preference accuracy. - [ ] I can compute sequence log-probs with the correct prompt mask and shift. - [ ] I can explain why Day 108's preference accuracy was 1.00 but exact match 0.17.

Extra notes

DPO is *offline*: it learns from a fixed set of pairs, usually generated by some other model. Online or iterative variants regenerate pairs from the current policy. Known DPO pathologies include the one seen here (both or only the rejected log-probs fall), sensitivity to ? and learning rate, and length exploitation. Variants add an NLL term on the chosen answer (tested on Day 112), normalise by length, or change the loss shape (IPO, KTO, ORPO, SimPO are names to look up; read their papers before relying on any claim about them).

Going deeper

- Read "Direct Preference Optimization: Your Language Model is Secretly a Reward Model" (Rafailov et al., 2023), sections 4 and 5, and match each equation to `day108_dpo.py`.
- Read the TRL library's DPO trainer documentation for what production code adds (padding, label masking, reference handling).

Interview-style question (15 min)

"Explain DPO to an engineer who knows SFT, and say when you would choose it over PPO." Points: same objective as KL-regularised RLHF; the optimal policy defines an implicit reward, so the BT loss can be written in policy log-probs; no RM or

sampling loop, so it is cheap and stable; but it is offline, depends on pair quality and coverage, and can lower the chosen likelihood. PPO/online methods are preferred when you have a good reward or verifier and want the model to learn from its own samples.

Common mistakes

- Including prompt tokens in the sequence log-prob.
- Using the policy (with gradients) as the reference.
- Averaging token log-probs in DPO but summing them for the reference (or vice versa).
- Reporting training preference accuracy as the result.

What to do next

Tomorrow: what ? and the learning rate change, and whether the model forgets its general language modelling.

Day 109: ?, learning rate and forgetting in DPO

Review today: Day 108 (+1), Day 106 (+3), Day 102 (+7), Day 88 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

? in DPO. In the RLHF objective ? is the KL weight. In the DPO loss it is the scale on log-ratios inside the sigmoid. Larger ? means a given log-prob change produces a bigger margin, so the loss saturates *sooner* and the gradient weight sigma(-margin) drops sooner. The policy therefore moves less from pi_ref. Smaller ? lets the policy move further before the loss is satisfied. That is the same direction as in Day 106: big ? = stay close.

Learning rate interacts. With LoRA at lr 5e-4 (Day 108) the optimiser can satisfy the loss in a few steps by drastic changes. Lower lr changes the path, not only the speed: a small step size finds changes closer to the reference.

Forgetting. Preference tuning can damage abilities that the pairs do not cover. A cheap proxy is perplexity on held-out general text. We reuse Week 15's Shakespeare validation text: 8 windows of 256 tokens, on which the instruct model scored 32.05 (Week 15 log `day104_eval_finetuned.log`).

Measuring movement. Three numbers tell you how far the policy moved: ?log-prob of chosen and rejected on held-out pairs, the change in general perplexity, and sample outputs. A full KL estimate would require sampling from the policy; ?log-probs on the pairs are a cheap substitute.

Block 2: Worked example (45 min)

`code/week-16/day109_dpo_sweep.py` reruns `day108_dpo.py` with lr 1e-4, 60 steps x 4 pairs, and ? in {0.1, 0.5}, and adds the Shakespeare perplexity.

Verified (out/day109_dpo_sweep.log):

Setting	Held-out margin	? chosen	? rejected	Exact match	Shakespeare ppl	Example output
Day 108: ? 0.1, lr 5e-4, 120 steps	13.53	+8.6	-126.7	0.17	not measured	"clouds orange springfirehose"
? 0.1, lr 1e-4, 60 steps	9.19	+10.7	-81.2	0.23	32.35	"clouds orange spring"
? 0.5, lr 1e-4, 60 steps	11.32	+12.3	-10.3	0.20	32.14	"cloud orange lemon spring"

Both lr 1e-4 runs trained in 35-36 s and reached held-out preference accuracy 1.00. With ? = 0.5 the rejected answer moved only -10.3 nats instead of -81.2, exactly the "stay closer" effect, and the example output keeps all four words (wrong order, but well-formed). Perplexity barely moved from 32.05 in either case (32.35 and 32.14), so on this proxy neither run forgot much. Exact match is 0.20-0.23 on 40 prompts; Day 110 shows that such differences are within noise.

Block 3: Lab (2h)

1. Run the sweep (about 2.5 minutes).
2. Add ? = 0.02 and ? = 2.0. Predict ? rejected before you run.
3. Run ? = 0.1 at lr 5e-4 and lr 1e-4 for the *same* 60 steps to separate the two effects.
4. Add a second forgetting probe: the 16-question multiple-choice set from Week 15.
5. Write a three-sentence recommendation for choosing ? on a new task.

Block 4: Practice (1h)

1. Why does larger ? make the loss saturate sooner?
2. Day 109 ? = 0.5: margin 11.32 with ? chosen +12.3 and ? rejected -10.3. Check the arithmetic.
3. Why is perplexity on Shakespeare only a proxy for forgetting?
4. Which setting would you pick from the table, and what would you need to be sure?

Answers. (1) The margin is ? times the log-ratio difference; the same log-prob change gives a ?-times larger margin. (2) 0.5 x (12.3 - (-10.3)) = 0.5 x 22.6 = 11.3, matching 11.32. (3) It measures one text domain and only likelihood, not instruction following, safety or reasoning. (4) None of the exact-match differences is reliable on n = 40; ? = 0.5 moves least and keeps outputs well-formed, so prefer it unless a larger test set shows otherwise.

Block 5: Review (1h)

Self-check: - [ ] I can predict the direction of the effect of ? on ? rejected. - [ ] I can explain why learning rate changes the solution, not just the speed. - [ ] I can measure forgetting with a held-out perplexity and say what it misses. - [ ] I can fill in and interpret the Day 109 table.

Extra notes

Changing ? and lr together is a common confounder (Week 15's LoRA vs full fine-tune comparison had the same problem). Change one thing at a time when you want to attribute an effect. Real runs also tune the number of epochs: DPO on a fixed dataset over-fits quickly, which in practice often shows as falling chosen log-probs.

Going deeper

- Search for analyses of "DPO chosen log-probability decrease" to see this effect discussed in the literature, and check whether the paper's setting matches yours.
- Read the IPO paper ("A General Theoretical Paradigm to Understand Learning from Human Preferences") for a loss that does not saturate in the same way.

Interview-style question (15 min)

"Your DPO run has 100% preference accuracy after 50 steps but the model got worse in human evals. What do you check?" Points: ?log-probs of chosen (falling?), output samples, length change, general benchmarks or perplexity for forgetting, ? and lr, number of epochs, the quality and diversity of the rejected answers, and the held-out set size.

Common mistakes

- Tuning ? while also changing the learning rate and steps.
- Comparing exact-match numbers that differ by 1-2 items out of 40.
- Not measuring anything outside the preference task.

What to do next

Tomorrow: a day on how evaluations mislead, measured on the Day 108 model.

Day 110: LLM evaluation pitfalls

Review today: Day 109 (+1), Day 107 (+3), Day 103 (+7), Day 89 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

An evaluation number is a measurement with error bars and assumptions. Six pitfalls cause most bad conclusions.

1. Small samples. With n test items and accuracy p, the standard error is about sqrt(p(1-p)/n). On 40 items at p = 0.2 that is about 0.06, so differences of 0.03-0.06 are noise. Report a confidence interval; the bootstrap (resample the per-item results with replacement many times) is simple and assumption-light.

2. Prompt sensitivity. Small models, and sometimes large ones, change their answers when the instruction is reworded, the casing changes or punctuation is removed. One prompt template is one sample from a distribution of reasonable prompts. Report several, or fix the template and say so.

3. Metric choice. Exact match, normalised match (lowercase, strip punctuation) and "contains the answer" can give different numbers for the same outputs. "Contains" can reward a model that outputs many candidates. Choose the metric before looking at results and report how it was computed.

4. Contamination. If test items (or close variants) appear in training data, scores measure memorisation. Check exact overlap and n-gram overlap. For public benchmarks and web-trained models you usually *cannot* fully check, which is why fresh, private or dynamically generated tests matter.

5. Judge bias. LLM judges have position, length and self-preference biases. Week 15 measured this: the 135M instruct judge gave 0 of 16 consistent verdicts when the two answers were swapped.

6. Optimising the metric. Once a number is a target, models and people tune toward it (Goodhart's law). The Day 107 RM shortcut and the Day 111 reward exploitation are versions of this.

Block 2: Worked example (45 min)

`code/week-16/day110_eval_pitfalls.py` loads the Day 108 DPO adapter and re-evaluates it on the 40 test prompts (15 s here).

Verified (out/day110_eval_pitfalls.log):

Prompt wording	Exact	Normalised	Contains
"Reverse the order of these words: ..." (training wording)	0.17	0.20	0.20
"reverse the order of these words ..." (lowercase, no colon)	0.07	0.07	0.07
"Write these words in reverse order: ..." (paraphrase)	0.12	0.12	0.12

- Bootstrap 95% CI for exact match with the training wording, n = 40: [0.07, 0.30] around 0.17.
- Contamination: 0/40 test prompts appear in training; 0/40 test word *sets* appear in training in another order.
- But 29/40 test answers have every word bigram somewhere in the training answers, so "no exact overlap" does not mean "nothing seen".

Reading: the same model scores between 0.07 and 0.20 depending on wording and metric, and the CI is wide enough to contain all of the Day 109 exact-match numbers (0.17-0.23). Any claim that one DPO setting is "better" on this test set would be unsupported.

Block 3: Lab (2h)

1. Run the script and compare with the log.
2. Raise the test set to 400 prompts (change `n_test`) and recompute the CI width. It should shrink roughly by sqrt10.
3. Add two more wordings; report mean and range across wordings.
4. Evaluate the Day 109 ? = 0.5 adapter the same way (save it with `TAG`). Is the difference with ? = 0.1 outside the CIs?
5. Implement a paired bootstrap: resample *items* and compute the difference between two models on the same items.
6. Write a short "evaluation card" for this test: data, size, metric, prompts, CI, known contamination.

Block 4: Practice (1h)

1. Model A scores 0.23 and model B 0.20 on the same 40 items. What can you conclude?
2. Why is a paired comparison more powerful than comparing two separate CIs?
3. Give an example where "contains" overstates performance.

4. Why might a model trained on one prompt template drop on a paraphrase?

Answers. (1) Nothing reliable: the difference is about one item, and Day 110's CI for one model spans [0.07, 0.30]. (2) Item difficulty is shared, so variance from "which items happened to be chosen" cancels. (3) A model that outputs both orders, or a list of guesses, contains the answer without giving it. (4) Fine-tuning on a narrow template teaches template-specific behaviour (measured: 0.17 -> 0.07 with lowercase and no colon).

Block 5: Review (1h)

Self-check: - [ ] I can compute a bootstrap CI and say what it covers and what it does not. - [ ] I can list six evaluation pitfalls with one example each. - [ ] I can run an exact and n-gram contamination check. - [ ] I can explain why "no exact overlap" is weaker than it sounds (29/40 bigram result).

Extra notes

The bootstrap CI covers sampling of *items* only. It says nothing about prompt choice, decoding settings, random seeds of training, or whether the test measures what you care about. Training-seed variance is its own source of noise: retrain with two seeds before trusting a small gain. Week 15's multiple-choice set gave every model the same score (0.88), which is the opposite pitfall: a test that cannot separate models.

Going deeper

- Read the "Holistic Evaluation of Language Models" (HELM) paper's discussion of prompt and metric choices.
- Search for papers on benchmark contamination detection and note which methods need access to training data.
- Read about Chatbot Arena's pairwise human-preference evaluation and its own biases.

Interview-style question (15 min)

"A new model beats the old one by 2 points on a public benchmark. What do you need before shipping?" Points: size of benchmark and CI or paired test, multiple prompts and seeds, contamination check, task-relevant private evals, failure analysis, safety checks, and whether the metric matches the product goal.

Common mistakes

- Reporting a single number with no n and no interval.
- Tuning prompts on the test set.
- Treating LLM-judge scores as ground truth.
- Assuming a held-out split is uncontaminated because exact strings do not match.

What to do next

Tomorrow: improving outputs at inference time by sampling several answers and choosing one, and what happens when the chooser is a bad reward.

Day 111: Best-of-n, verifiers and exploiting an implicit reward

Review today: Day 110 (+1), Day 108 (+3), Day 104 (+7), Day 90 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Best-of-n (rejection sampling). Sample n answers, score each, return the best. It is the simplest way to "optimise" against a reward without changing weights. Its strength depends on two things: whether the right answer appears among the samples at all (*pass@n*), and whether the scorer can recognise it.

Scorers. (a) The policy's own log-prob: picks the answer the model already likes most; close to greedy. (b) A learned reward model: works if the RM is accurate on these samples. (c) A *verifier*: for tasks with checkable answers (unit tests, exact arithmetic, word reversal) a program can decide correctness. With a perfect verifier, best-of-n accuracy equals pass@n.

DPO's implicit reward as a scorer. DPO defines r?(x,y) = ?.log(pi_?(y|x)/pi_ref(y|x)). It looks like a free reward model. But it was only trained to rank the chosen/rejected kinds of answers. On other samples, especially strange ones the reference finds very unlikely, the ratio can be huge for a bad answer. That is reward hacking at inference time.

Best-of-n and RLHF. Best-of-n is also used to build training data (keep the best samples and fine-tune on them: "rejection sampling fine-tuning", or "expert iteration"), and as a baseline: an RL method should beat best-of-n at the same compute.

Block 2: Worked example (45 min)

`code/week-16/day111_best_of_n.py`: for each of the 40 test prompts, sample 8 answers from the Day 108 DPO model at temperature 0.8 and choose by each scorer. The implicit reward uses ? = 0.1; the reference log-probs come from the same model with the LoRA scale set to 0. Run time 54 s.

Verified (out/day111_best_of_n.log): - Greedy: 0.17. - Best-of-8 by policy log-prob: 0.15. - Best-of-8 by DPO implicit reward: **0.00**. - Oracle verifier (pass@8): 0.20. - Mean distinct samples per prompt: 5.4 of 8.

The verifier gains only a little over greedy (0.17 -> 0.20), because the correct answer rarely appears among 8 samples: this policy is weak, so selection cannot help much. Choosing by the implicit reward never picked a correct answer, even on prompts where the verifier found one among the samples (random choice was not measured). The samples it picks are ones the reference model finds extremely unlikely, which the log-ratio rewards. A scorer outside its training distribution is exploited by optimisation, even one as mild as picking 1 of 8.

Block 3: Lab (2h)

1. Run the script and compare with the log.
2. Print the chosen sample for the implicit-reward scorer on five prompts. Describe what they have in common.
3. Use length-normalised implicit reward (divide by the number of answer tokens). Does it still score 0.00?
4. Try n = 2, 4, 16 and plot pass@n and best-of-n by policy log-prob against n.
5. Use the Week 15 SFT adapter (`code/week-15/out/day103_lora.pt`) as the policy. With a model that already solves the task, what does pass@8 become?

Block 4: Practice (1h)

1. Why can best-of-n with a perfect verifier never exceed pass@n?
2. Why might choosing by the policy's log-prob (0.15) be slightly below greedy (0.17)?
3. Why is the DPO implicit reward exploitable on samples it never saw?
4. Name two tasks where verifiers exist and two where they do not.

Answers. (1) It can only return one of the samples; if none is correct, nothing is. (2) Sampling at temperature 0.8 may not include the greedy answer, and the highest-probability *sample* is not always the greedy sequence; with n = 40 the 0.02 gap is one item, i.e. noise. (3) It is a log-ratio; any answer the reference rates very unlikely gets a big value if the policy rates it less unlikely, whether or not it is good. (4) Verifiable: code with tests, maths with known answers. Not verifiable: essay quality, helpfulness of advice.

Block 5: Review (1h)

Self-check: - [ ] I can define pass@n and best-of-n and explain how they relate. - [ ] I can compare three scorers and explain the Day 111 numbers. - [ ] I can explain why an implicit or learned reward breaks under selection pressure. - [ ] I know where best-of-n is used in training pipelines.

Extra notes

Best-of-n has a cost (n generations per query) and a well-known analytic KL to the base policy, which grows only like log n. That makes it a useful reference point when comparing RL methods. Verifier-based methods (reinforcement learning with verifiable rewards) are a major current direction exactly because verifiers do not drift like learned RMs; but they exist only for some tasks.

Going deeper

- Search for "Let's Verify Step by Step" (process vs outcome reward models).

- Read about the pass@k estimator in the Codex paper ("Evaluating Large Language Models Trained on Code") and why it uses an unbiased estimator rather than taking k samples once.

Interview-style question (15 min)

"You have a code model and unit tests. Would you use best-of-n, RL, or both?" Points: unit tests are a verifier, so best-of-n at inference gives pass@n directly at nx cost; RL with test-based rewards can raise pass@1 so fewer samples are needed; tests can be gamed (special-casing, weak tests), so hold out tests; compare RL against best-of-n at equal compute.

Common mistakes

- Using the DPO implicit reward as a general reward model without checking it.
- Reporting pass@n as if it were single-answer accuracy.
- Forgetting the nx inference cost.

What to do next

Tomorrow's project: the same pairs, three objectives (SFT, DPO, DPO+NLL), one table.

Day 112: PROJECT W16: SFT vs DPO vs DPO+NLL; Quiz 16

Review today: Day 111 (+1), Day 109 (+3), Day 105 (+7), Day 91 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

The question. Given preference pairs, is it better to (a) do SFT on the chosen answers only, (b) do DPO, or (c) do DPO plus an NLL (SFT) term on the chosen answer? Option (c) is a common fix for DPO lowering the chosen likelihood: the NLL term explicitly keeps the chosen answer probable.

When does DPO add something over SFT? DPO uses information SFT ignores: *what is wrong*. That matters when the chosen and rejected answers are both plausible and differ in a subtle way, and when the chosen answers are not good enough to imitate outright but are better than the alternative. When the chosen answer is simply the correct answer (as in our synthetic task), SFT gets the full signal and DPO is mostly a less direct way to reach it.

Fair comparison. Same 240 pairs, same LoRA (r=16, all linear), same lr (1e-4), same 60 steps x 4 pairs, same 40 test prompts, same metric (exact match, greedy, training wording) and the same forgetting probe (Shakespeare ppl, instruct reference 32.05). The DPO numbers come from the Day 109 run with these settings (? 0.1).

Project steps. 1. Run `code/week-16/day112_project.py` (SFT and DPO+NLL; about 1.5 minutes). 2. Put the Day 109 ? = 0.1 result beside it. 3. Add the Day 110 lesson: compute or at least state the uncertainty. 4. Write a short report: question, setup, table, interpretation, limitations, what a GPU would change.

Block 2: Worked example (45 min)

Verified (out/day112_project.log and out/day109_dpo_sweep.log):

Objective (60 steps x 4 pairs, lr 1e-4, LoRA r=16)	Train time	Exact match (n = 40)	Shakespeare ppl	Example output for "spring orange lemon cloud"
SFT on chosen	18 s	**0.70**	32.13	"cloud lemon orange spring" (correct)
DPO, ? 0.1 (Day 109)	36 s	0.23	32.35	"clouds orange spring"
DPO + NLL on chosen, ? 0.1	35 s	**0.70**	32.17	"cloud lemon orange spring" (correct)

SFT and DPO+NLL both reach 0.70 and keep perplexity near the instruct model's 32.05. Plain DPO reaches only 0.23. Here the difference between 0.70 and 0.23 is about 19 items out of 40, far outside the noise level measured on Day 110. SFT is also twice as fast per step, because DPO needs the rejected answer too. Adding DPO to SFT did not help on this task (0.70 vs 0.70), which fits the concept lesson: our chosen answers are exactly correct, so imitation captures the full signal.

Block 3: Lab (2h)

Write the project report (`code/week-16/out/report.md` or your notes) with: 1. The table above, reproduced from your own run. 2. A paired bootstrap CI for SFT - DPO (Day 110 lab 5). 3. One experiment of your choice: (a) make chosen answers only *partly* correct (e.g. right order but one word misspelled) and see whether DPO+NLL beats SFT; (b) use 120 steps; (c) vary the NLL weight (0.1, 1, 10). 4. Limitations: synthetic task, one seed, 40 test items, 135M model, CPU-sized budget. 5. What a GPU would change: thousands of real pairs, 1-8B models, multiple seeds, online DPO or PPO with a learned RM, and human or held-out-judge evaluation.

Block 4: Practice (1h)

1. Why is SFT twice as fast per step as DPO here?
2. Why would you expect DPO to help more when chosen answers are imperfect?
3. Why is the NLL term a natural fix for Day 108's falling-mass problem?
4. Name one thing this project cannot tell you about real preference tuning.

Answers. (1) Each DPO step processes the chosen and rejected sequences; SFT processes only the chosen (measured 18 s vs 35-36 s). (2) SFT would copy their flaws; DPO learns only the *direction* "more like chosen, less like rejected". (3) It directly rewards high likelihood for the chosen answer, so the loss cannot be satisfied by only lowering the rejected answer. (4) Behaviour with noisy, subjective human preferences at scale; anything about large models.

Block 5: Review (1h)

Self-check: - [ ] I can design a fair comparison of preference-tuning objectives. - [ ] I can reproduce the project table and explain each row. - [ ] I can say when DPO should and should not beat SFT. - [ ] I completed Quiz 16 below.

Extra notes

This week's honest summary: on a clean synthetic task with a 135M model, preference accuracy was a poor guide to quality; DPO alone was worse than SFT; DPO+NLL matched SFT; reward models and implicit rewards were exploitable. These are known qualitative patterns, reproduced at toy scale; the specific numbers say nothing about large models.

Going deeper

- Read the Zephyr technical report for a real SFT-then-DPO recipe with AI-labelled preferences (search by title).
- Search the TRL documentation for the DPO trainer's options for adding an SFT loss term and compare with `day112_project.py`.

Interview-style question (15 min)

"You have 10k preference pairs. Do you SFT on the chosen answers, run DPO, or both?" Points: start with SFT on chosen (cheap, strong baseline); add DPO, ideally with an NLL term, when the rejected answers carry information and chosen ones are not ideal; evaluate both with CIs, forgetting probes and human or judge checks; watch chosen log-probs and length.

Common mistakes

- Comparing objectives with different steps, learning rates or data.
- Leaving SFT out as a baseline.
- Claiming one method is better from a 1-2 item difference.

What to do next

Week 17 continues Month 4 with the topics moved out of this week (see `SYLLABUS.md`).

Weekly quiz: Week 16

1. Name the three stages of the classic RLHF pipeline.
2. Write the Bradley-Terry probability that y_w is preferred to y_l.

3. Why is a reward model's output defined only up to a constant per prompt?
4. What is the closed-form optimum of the KL-regularised objective for one prompt?
5. In Day 106's bandit, what happened to expected reward and KL as ? went from 5.0 to 0.05?
6. Why is the reward read from the last token in a causal-LM reward model?
7. What did RM B (chosen always ending with "!") score when the "!" was moved to the wrong answer, and what does that show?
8. Write the DPO loss.
9. On Day 108, preference accuracy was 1.00 but exact match 0.17. Explain.
10. What did increasing ? from 0.1 to 0.5 do to ?log-prob of the rejected answer on Day 109?
11. Why was the DPO implicit reward a bad best-of-n scorer on Day 111?
12. On Day 112, which objectives reached 0.70 exact match, and why did plain DPO not help on this task?

Answers

1. SFT on demonstrations; reward model from pairwise preferences; policy optimisation (e.g. PPO) against the RM with a KL penalty to the SFT/reference model.
2. P = sigma(r(x, y_w) - r(x, y_l)).
3. BT uses only reward differences, so adding a constant changes nothing (Day 106 measured a 0.0e+00 change in loss).
4. pi*(y) ? pi_ref(y).exp(r(y)/?).
5. Expected reward rose from 0.309 to 0.993 and KL to the reference rose from 0.002 to 2.816.
6. In a causal model only the last position has attended to the entire prompt and answer.
7. 0.00 pairwise accuracy: it learned the planted shortcut rather than correctness (also only 0.62 with no "!" at all).
8. L = -log sigma(?[(log pi_?(y_w|x) - log pi_ref(y_w|x)) - (log pi_?(y_l|x) - log pi_ref(y_l|x))]).
9. DPO satisfied the loss mainly by pushing the rejected answer down (? -126.7 nats held-out) while raising the chosen one only a little (+8.6); that does not make the correct answer the greedy output, and outputs degraded.
10. It shrank from -81.2 to -10.3 nats: larger ? keeps the policy closer to the reference.
11. It is a log-ratio trained only on chosen/rejected kinds of answers; on new samples it favoured ones the reference found very unlikely, scoring 0.00 vs 0.20 for the verifier.
12. SFT on chosen and DPO+NLL both reached 0.70 (plain DPO 0.23). The chosen answers were exactly correct, so imitation already captured the full signal; the NLL term stopped DPO from merely lowering the rejected answer.